

MDS5112 - Deep Learning and Applications

Assignment 1

March 16, 2025

Name: Zijin CAI
Student ID: 224040002

1 Task A - Build a Neural Network Using PyTorch

This section presents the implementation of neural networks using `pytorch`, to classify images from the *CIFAR-10* and *MNIST* datasets, with experiments conducted concerning different factors to enhance the model's performance. The code snippets provided below demonstrate the environment setup, data loading, model definition, training loop, and evaluation processes. The experiments are conducted on the *CIFAR-10* dataset, include a baseline CNN model, improved CNN models with batch normalization, data augmentation, deeper architectures, and dropout layers. The best performance model is then applied to the *MNIST* dataset to evaluate, respectively a different dataset.

1.1 Environment Setup and Reproducibility

With the proper virtual environment activated (`venv` or `conda env`), install the required package:

```
1 $ pip install torch==2.6.0 torchvision==0.21.0
```

Run `pip check` to make sure no broken dependencies, and setting up the foundational environment by importing essential libraries, including `pytorch` for neural network operations, `numpy` for numerical computations, and `matplotlib` for visualization. Checks for NVIDIA GPU availability and sets the device to use `cuda` if available. To ensure reproducibility, random seeds are set for `pytorch`, `cuda`, `numpy`, and `python` using a specific seed value (e.g., student ID). Additionally, the `torch.backends.cudnn.deterministic` flag is enabled to ensure consistent results across runs. This setup is crucial for creating a stable and reproducible training environment for deep learning experiments.

```
1 # Import the necessary libraries
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5 import torch.optim as optim
6 import torchvision
7 import torchvision.transforms as transforms
8 from torch.utils.data import DataLoader
9 import numpy as np
10 import random
11 import matplotlib.pyplot as plt
12
13 # Set the device to GPU if available
14 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
15
16 # Set the random seed for reproducibility
17 seed = 224040002 # any number (i.e., student ID)
18 torch.manual_seed(seed) # Set the random seed for PyTorch
19 torch.cuda.manual_seed(seed) # Set the random seed for CUDA
20 np.random.seed(seed) # Set the random seed for NumPy
21 random.seed(seed) # Set the random seed for Python
22 torch.backends.cudnn.deterministic = True # Set the random seed for all devices
```

1.2 Load and Preprocess the Datasets

load the *CIFAR-10* and *MNIST* datasets with necessary transformations (`./data` is the directory where the datasets are stored), then define the data preprocessing pipeline and load the datasets with basic transformations for each using `transforms.Compose`. For *CIFAR-10*, the images are converted to pytorch tensors and normalized using mean and standard deviation values of 0.5 for all three channels (RGB). For *MNIST*, the images are similarly converted to tensors and normalized, but with a single channel (grayscale). The datasets are then loaded using `torchvision.datasets`, with the training and testing sets downloaded and transformed accordingly. Data loaders are created for both datasets using `DataLoader`, with a batch size of 32 and shuffling enabled for the training sets. This setup ensures the data is properly prepared and efficiently fed into the model during training and evaluation.

```

1 # Define basic transformations for the datasets
2 transform_cifar = transforms.Compose([
3     transforms.ToTensor(), # Convert the image to a PyTorch tensor
4     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize the image
5 ])
6
7 transform_mnist = transforms.Compose([
8     transforms.ToTensor(), # Convert the image to a PyTorch tensor
9     transforms.Normalize((0.5,), (0.5,)) # Normalize the image
10 ])
11
12 # Load datasets
13 trainset_cifar = torchvision.datasets.CIFAR10(root='./data', train=True, download=
14     True, transform=transform_cifar)
15 testset_cifar = torchvision.datasets.CIFAR10(root='./data', train=False, download=
16     True, transform=transform_cifar)
17
18 trainset_mnist = torchvision.datasets.MNIST(root='./data', train=True, download=True,
19     transform=transform_mnist)
20 testset_mnist = torchvision.datasets.MNIST(root='./data', train=False, download=True,
21     transform=transform_mnist)
22
23 # Create data loaders
24 trainloader_cifar = DataLoader(trainset_cifar, batch_size=32, shuffle=True)
25 testloader_cifar = DataLoader(testset_cifar, batch_size=32, shuffle=False)
26
27 trainloader_mnist = DataLoader(trainset_mnist, batch_size=32, shuffle=True)
28 testloader_mnist = DataLoader(testset_mnist, batch_size=32, shuffle=False)

```

1.3 Experiments with the CIFAR-10 Dataset

To improve accuracy, the experiments are conducted with several key factors:

- **Batch Normalization:** to stabilize and accelerate training.
- **Data Augmentation:** to enhance generalization by exposing the model to more varied data.
- **Deeper Architectures:** to capture more complex features with deeper neural networks.
- **Dropout Layer:** to drop out some neurons during training thus prevent overfitting.

The following table summarizes the experiments conducted and the test accuracies achieved:

Experiment	Test Accuracy	Implementation Details
Baseline CNN	71%	Simple architecture of Baseline
+ BatchNorm	73%	Stabilized training and performance
+ Data Augmentation	78%	Reduced overfitting with generalization
+ Deeper + Dropout	83%	Enhanced feature learning and regularization
Ultimate Example Baseline	87%	Combination of above improvements baseline
Ultimate Enhanced Model	90%	Model selection with further enhancements

Table 1: Summary of experiments and test accuracies on the CIFAR-10 dataset.

Baseline Model with a Simple CNN

This section of code presents a baseline CNN model designed for image classification on the *CIFAR-10* dataset. The `BaselineCNN` consists of two convolutional layers followed by max-pooling and two fully connected layers. The convolutional layers extract features from the input images, while the fully connected layers classify these features into one of the 10 output classes. The model uses the `ReLU` activation function for non-linearity and max-pooling for downsampling. It is trained using the `Adam` optimizer with a learning rate of 0.001 and the `CrossEntropyLoss` function. This baseline model serves as a starting point for further improvements and experimentation, and the test accuracy achieved by the baseline model is around 71% with the loss around 0.06. (Figure 1)

```

1 class BaselineCNN(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = nn.Conv2d(3, 32, 3, padding=1) # CIFAR-10 has 3 input channels
5         self.conv2 = nn.Conv2d(32, 64, 3, padding=1) # 32 input channels
6         self.fc1 = nn.Linear(64 * 8 * 8, 256) # 64 channels, 8x8 image size
7         self.fc2 = nn.Linear(256, 10) # 10 output classes
8
9     def forward(self, x):
10        x = F.relu(self.conv1(x)) # Apply the ReLU activation function
11        x = F.max_pool2d(x, 2) # Apply max pooling with a 2x2 kernel
12        x = F.relu(self.conv2(x)) # Apply the ReLU activation function
13        x = F.max_pool2d(x, 2) # Apply max pooling with a 2x2 kernel
14        x = torch.flatten(x, 1) # Flatten the output of the convolutional layers
15        x = F.relu(self.fc1(x)) # Apply the ReLU activation function
16        x = self.fc2(x)
17        return x
18
19 # Define the model, loss function, and optimizer
20 model = BaselineCNN().to(device) # Move the model to the GPU if available
21 criterion = nn.CrossEntropyLoss() # Define the loss function
22 optimizer = optim.Adam(model.parameters(), lr=0.001) # Define the optimizer
23
24 for epoch in range(num_epochs):
25     model.train() # Set the model to training mode
26     running_loss = 0.0
27     for i, data in enumerate(trainloader_cifar):
28         inputs, labels = data
29         inputs, labels = inputs.to(device), labels.to(device)
30         optimizer.zero_grad() # Zero the parameter gradients
31         outputs = model(inputs) # Forward pass
32         loss = criterion(outputs, labels) # Calculate the loss
33         loss.backward() # Backward pass
34         optimizer.step() # Optimize
35         running_loss += loss.item() # Add the loss
36
37     losses.append(running_loss / len(trainloader_cifar)) # Calculate the average loss
38
39 # Evaluate the model
40 model.eval() # Set the model to evaluation mode
41 correct = 0
42 total = 0
43 with torch.no_grad():
44     for data in testloader_cifar:
45         inputs, labels = data
46         inputs, labels = inputs.to(device), labels.to(device)
47         outputs = model(inputs) # Forward pass
48         _, predicted = torch.max(outputs.data, 1) # Get the predicted class
49         total += labels.size(0) # Update the total count
50         correct += (predicted == labels).sum().item() # Update the correct count
51
52 accuracies.append(100 * correct / total) # Calculate the test accuracy
53 print(f'Epoch {epoch + 1}, Loss: {running_loss / len(trainloader_cifar)},
54       Accuracy: {100 * correct / total}%')
```

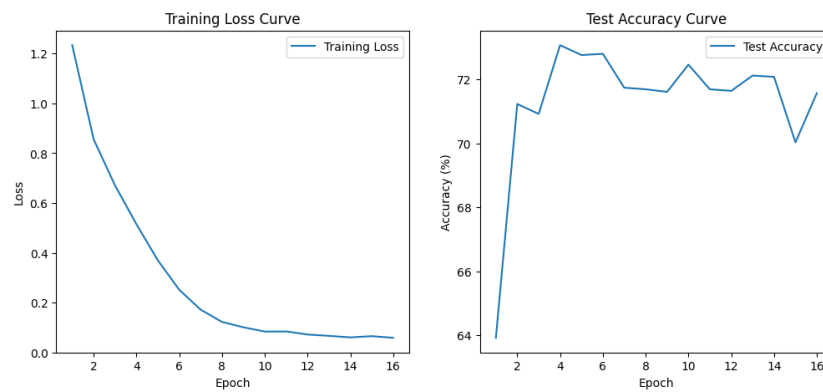


Figure 1: Training loss and test accuracy of the baseline CNN model on the CIFAR-10 dataset.

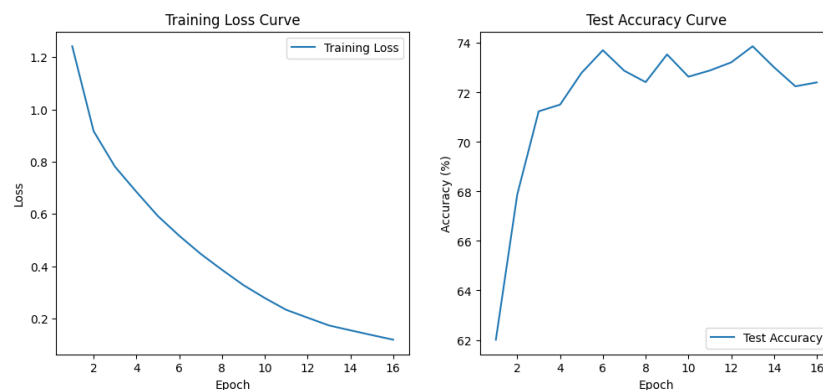


Figure 2: Training loss and test accuracy of the improved CNN model with batch normalization.

Improved Model with Batch Normalization

This section introduces an improved CNN model that incorporates batch normalization to enhance training stability and performance. The ImprovedCNN includes two convolutional layers with batch normalization applied after each convolution. Batch normalization helps stabilize the learning process by normalizing the inputs of each layer, which can lead to faster convergence and better generalization. The test accuracy achieved by threshold the improved model with batch normalization is around 73%. (Figure 2)

```

1 # Improved model with batch normalization
2 class ImprovedCNN(nn.Module):
3     def __init__(self):
4         super().__init__()
5         self.conv1 = nn.Conv2d(3, 32, 3, padding=1) # 3 input, 32 output channels
6         self.bn1 = nn.BatchNorm2d(32) # Batch normalization stabilizes the learning
7         self.conv2 = nn.Conv2d(32, 64, 3, padding=1) # 32 input, 64 output channels
8         self.bn2 = nn.BatchNorm2d(64) # Batch normalization stabilizes the learning
9         # ... rest of the layers
10
11     def forward(self, x):
12         x = F.relu(self.bn1(self.conv1(x))) # Apply batch normalization
13         # ... rest of the forward pass
14
15 # Define the model, loss function, and optimizer
16 model = ImprovedCNN().to(device) # Move the model to the GPU if available
17 criterion = nn.CrossEntropyLoss() # Define the loss function
18 optimizer = optim.Adam(model.parameters(), lr=0.001) # Define the optimizer
19
20 num_epochs = 32 # Define the number of epochs
21 # ... rest of the training loop

```

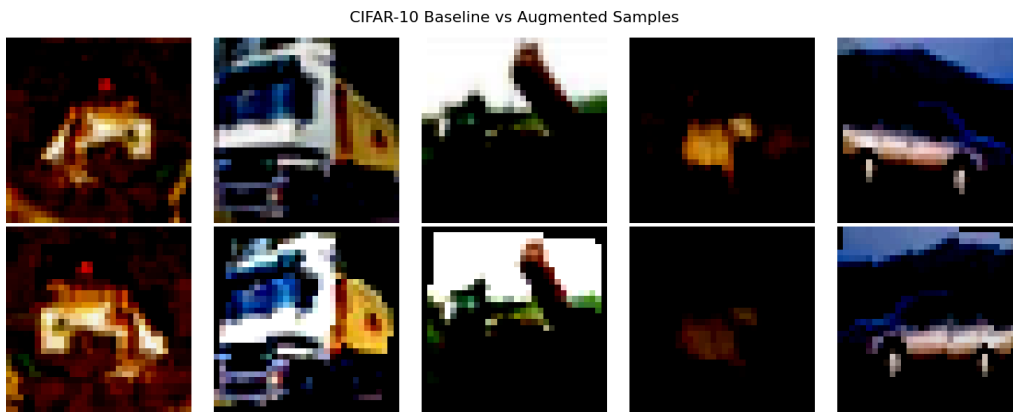


Figure 3: Sample images from the CIFAR-10 dataset with data augmentation.

Updated CIFAR-10 Transform with Data Augmentation

This section of code introduces data augmentation techniques to improve the generalization ability of the CNN model on the *CIFAR-10* dataset. The `transforms.Compose` define a series of transformations, including random horizontal flips, rotations, affine transformations (shear and scale), and color jittering (brightness, contrast, and saturation adjustments). The following examples are sample images applied with data augmentation techniques (Figure 3). These augmentations help the model become more robust by exposing it to a wider variety of training data. The images are then converted to pytorch tensors and normalized, trained with the `ImprovedCNN` model. The test accuracy achieved by threshold the improved model with data augmentation is around 78%. (Figure 4)

```

1  transform = transforms.Compose([
2      transforms.RandomHorizontalFlip(), # Randomly flip the image horizontally
3      transforms.RandomRotation(10), # Randomly rotate the image by 10 degrees
4      transforms.RandomAffine(0, shear=10, scale=(0.8, 1.2)), # Affine transformation
5      transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), # Randomly
        change the brightness, contrast, and saturation
6      transforms.ToTensor(), # Convert the image to a PyTorch tensor
7      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize the image
8  ])
9
10 # Load the CIFAR-10 dataset with the updated transform
11 trainset_cifar = torchvision.datasets.CIFAR10(root='./data', train=True, download=
    True, transform=transform)
12 trainloader_cifar = DataLoader(trainset_cifar, batch_size=32, shuffle=True)
13
14 # Train the model with the updated transform
15 model = ImprovedCNN().to(device) # Move the model to the GPU if available
16 criterion = nn.CrossEntropyLoss() # Define the loss function
17 optimizer = optim.Adam(model.parameters(), lr=0.001) # Define the optimizer
18
19 num_epochs = 32 # Define the number of epochs
20 losses = [] # Store the loss values
21 accuracies = [] # Store the test accuracies
22
23 for epoch in range(num_epochs):
24     # ... rest of the training loop

```

Deeper Neural Network with Dropout Layer

This section defines a `DeeperCNN` model, incorporating the dropout layers and batch normalization to enhance model performance and prevent overfitting. The model includes three convolutional layers, each followed by batch normalization and max-pooling, to extract hierarchical features from the input images. After the convolutional layers, the output is flattened and passed through two fully connected layers, with a dropout layer applied to the first fully connected layer to regularize the model. Dropout randomly deactivates 20% of the neurons during training, encouraging the network to learn more robust features. The final test accuracy threshold achieved by the deeper model with a dropout layer is around 83%. (Figure 5)

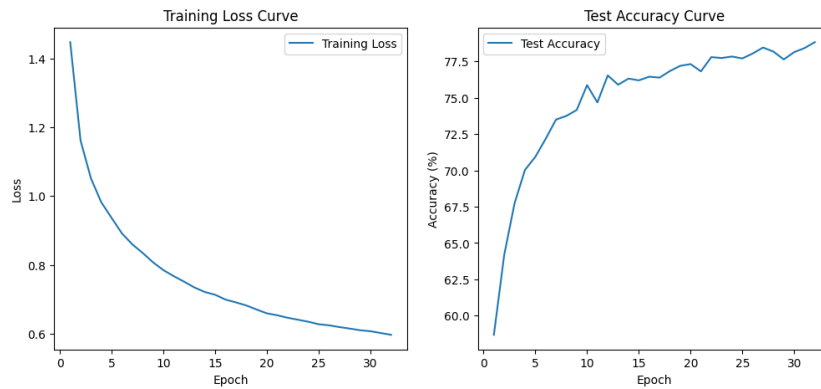


Figure 4: Training loss and test accuracy of the improved CNN model with data augmentation.

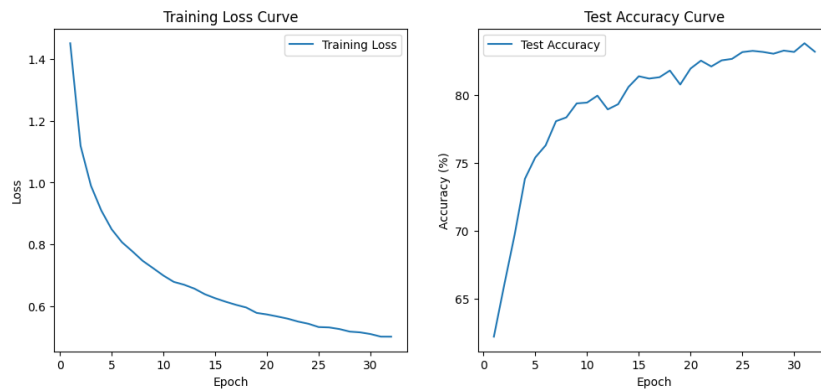


Figure 5: Training loss and test accuracy of the deeper CNN model with a dropout layer.

```

1 class DeeperCNN(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = nn.Conv2d(3, 32, 3, padding=1) # 3 input, 32 output channels
5         self.bn1 = nn.BatchNorm2d(32) # Batch normalization stabilizes the learning
6         self.conv2 = nn.Conv2d(32, 64, 3, padding=1) # 32 input, 64 output channels
7         self.bn2 = nn.BatchNorm2d(64) # Batch normalization stabilizes the learning
8         self.conv3 = nn.Conv2d(64, 128, 3, padding=1) # 64 input, 128 output channels
9         self.bn3 = nn.BatchNorm2d(128) # Batch normalization stabilizes the learning
10        self.fc1 = nn.Linear(128 * 4 * 4, 512) # 128 channels, 4x4 image size
11        self.fc2 = nn.Linear(512, 10) # 10 output classes
12        self.dropout = nn.Dropout(0.2) # Apply dropout with a probability of 0.2
13
14    def forward(self, x):
15        x = F.relu(self.bn1(self.conv1(x))) # Apply batch normalization
16        x = F.max_pool2d(x, 2) # Apply max pooling with a 2x2 kernel
17        x = F.relu(self.bn2(self.conv2(x))) # Apply batch normalization
18        x = F.max_pool2d(x, 2) # Apply max pooling with a 2x2 kernel
19        x = F.relu(self.bn3(self.conv3(x))) # Apply batch normalization
20        x = F.max_pool2d(x, 2) # Apply max pooling with a 2x2 kernel
21        x = torch.flatten(x, 1) # Flatten the output of the convolutional layers
22        x = F.relu(self.fc1(x)) # Apply the ReLU activation function
23        x = self.dropout(x) # Apply dropout
24        x = self.fc2(x)
25        return x
26
27 num_epochs = 32 # Define the number of epochs
28 losses = [] # Store the loss values
29 accuracies = [] # Store the test accuracies
30 # ... rest of the training loop

```

1.4 Ultimate Model Selection on CIFAR-10: Example Baseline and Enhancement

The ultimate model selection on the *CIFAR-10* dataset involves comparing the baseline model with the further enhanced models incorporating batch normalization, data augmentation, more architectural depth and dropout layers. This section presents two models, the **Example Baseline Model** is a refined version of the baseline model with additional layers and regularization techniques, while the **Enhanced CNN Model** is a more sophisticated architecture that leverages batch normalization, dropout, and data augmentation to improve performance.

```

1 # Define the ultimate data augmentation transform
2 # One could be identical to the previous as already includes:
3 # random horizontal flips, rotations, affine transformations, and color jittering
4 transform_cifar = transforms.Compose([
5     transforms.RandomHorizontalFlip(), # Randomly flip the image horizontally
6     transforms.RandomRotation(10), # Randomly rotate the image by 10 degrees
7     transforms.RandomAffine(0, shear=10, scale=(0.8, 1.2)), # Affine transformation
8     transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), # Randomly
9     # change the brightness, contrast, and saturation
10    transforms.ToTensor(), # Convert the image to a PyTorch tensor
11    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize the image
12 ])

```

The **Example Baseline Model** consists four convolutional layers, each followed by a **ReLU** activation function and a max-pooling layer to extract and downsample features. To prevent overfitting, dropout layers are strategically placed after each max-pooling operation and before the fully connected layers. After the convolutional and pooling layers, the output is flattened and passed through three fully connected layers, with dropout applied after each to further regularize the model. The final layer is a linear layer that outputs the classification probabilities for 10 classes. This architecture is designed to learn robust features from the input images and improve the model's generalization capabilities. The final test accuracy threshold achieved by the model is around 87% (Figure 6).

```

1 trainset_cifar = torchvision.datasets.CIFAR10(root='./data',
2                                             train=True, download=True, transform=transform)
3 trainloader_cifar = DataLoader(trainset_cifar,
4                                batch_size=64, shuffle=True, num_workers=2)
5
6 # Define the baseline model from the example code
7 ExampleBaselineCNN = nn.Sequential(
8     nn.Conv2d(3, 128, 3, padding=1), nn.ReLU(inplace=True),
9     nn.MaxPool2d(2), nn.Dropout(0.3),
10    nn.Conv2d(128, 256, 3, padding=1), nn.ReLU(inplace=True),
11    nn.MaxPool2d(2), nn.Dropout(0.3),
12    nn.Conv2d(256, 512, 3, padding=1), nn.ReLU(inplace=True),
13    nn.Conv2d(512, 512, 3, padding=1), nn.ReLU(inplace=True),
14    nn.Conv2d(512, 256, 3, padding=1), nn.ReLU(inplace=True),
15    nn.MaxPool2d(2), nn.Dropout(0.3),
16    nn.Flatten(),
17    nn.Linear(256 * 4 * 4, 512), nn.ReLU(inplace=True), nn.Dropout(0.5),
18    nn.Linear(512, 256), nn.ReLU(inplace=True), nn.Dropout(0.5),
19    nn.Linear(256, 128), nn.ReLU(inplace=True), nn.Dropout(0.5),
20    nn.Linear(128, 10),
21 )
22
23 # Define the model, loss function, and optimizer
24 model = ExampleBaselineCNN.to(device) # Move the model to the GPU if available
25 criterion = nn.CrossEntropyLoss() # Define the loss function
26 optimizer = optim.Adam(model.parameters(), lr=3e-4, weight_decay=1e-6)
27
28 num_epochs = 128 # Define the number of epochs
29 # ... rest of the training loop

```

The **Enhanced CNN Model** employs a series of convolutional layers, each with an increasing number of filters to capture more complex features, followed by batch normalization to stabilize learning and accelerate convergence. **ReLU** activation functions are utilized to introduce non-linearity, while max-pooling layers downsample the feature maps, reducing spatial dimensions and computational load. Dropout layers are strategically incorporated to mitigate overfitting by randomly deactivating a fraction of neurons during training. The final layer is a linear layer that outputs the classification probabilities for 10 classes. The final test accuracy surpass the baseline achieved around 90%.

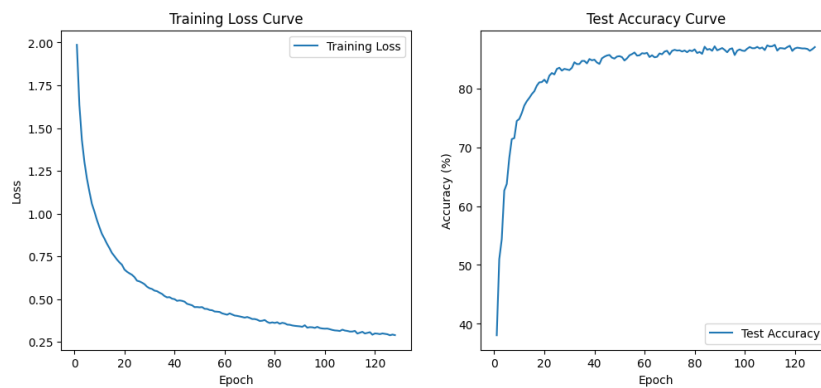


Figure 6: Training loss and test accuracy of the example baseline CNN model.

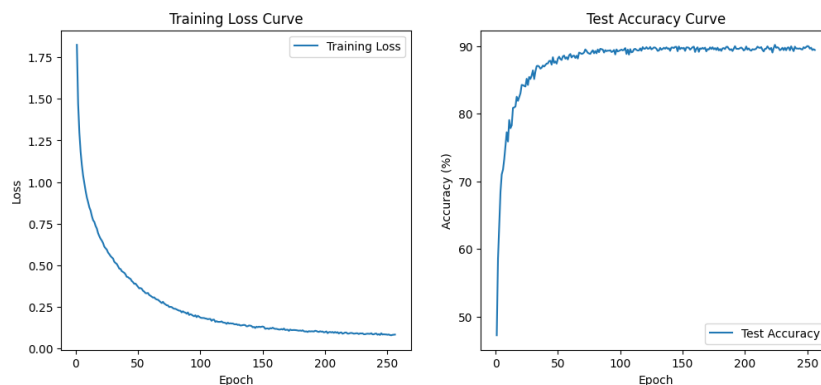


Figure 7: Training loss and test accuracy of the enhanced CNN model.

```

1 trainset_cifar = torchvision.datasets.CIFAR10(root='./data',
2                                             train=True, download=True, transform=transform)
3 trainloader_cifar = DataLoader(trainset_cifar,
4                                batch_size=64, shuffle=True, num_workers=2)
5
6 # Define the ultimate enhanced model
7 OptimizedCNN = nn.Sequential(
8     nn.Conv2d(3, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(inplace=True),
9     nn.MaxPool2d(2, 2), nn.Dropout(0.3),
10    nn.Conv2d(128, 256, 3, padding=1), nn.BatchNorm2d(256), nn.ReLU(inplace=True),
11    nn.MaxPool2d(2, 2), nn.Dropout(0.3),
12    nn.Conv2d(256, 512, 3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
13    nn.Conv2d(512, 512, 3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
14    nn.Conv2d(512, 256, 3, padding=1), nn.BatchNorm2d(256), nn.ReLU(inplace=True),
15    nn.MaxPool2d(2, 2), nn.Dropout(0.3),
16    nn.Flatten(),
17    nn.Linear(256 * 4 * 4, 512), nn.ReLU(inplace=True), nn.Dropout(0.5),
18    nn.Linear(512, 256), nn.ReLU(inplace=True), nn.Dropout(0.5),
19    nn.Linear(256, 128), nn.ReLU(inplace=True), nn.Dropout(0.5),
20    nn.Linear(128, 10)
21 )
22
23 # Define the model, loss function, and optimizer
24 model = ExampleBaselineCNN.to(device) # Move the model to the GPU if available
25 criterion = nn.CrossEntropyLoss() # Define the loss function
26 optimizer = optim.Adam(model.parameters(), lr=3e-4, weight_decay=1e-6)
27
28 num_epochs = 256 # Define the number of epochs
29
30 for epoch in range(num_epochs):
31     # ... rest of the training loop

```

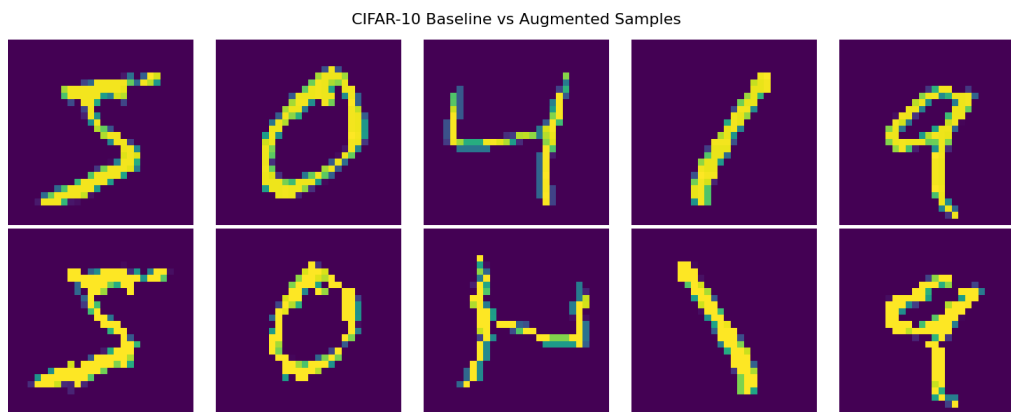



Figure 8: Sample images from the MNIST dataset with data augmentation.

1.5 Apply the Best Model to the MNIST Dataset

This section defines a `MNIST_CNN` model specifically designed for the *MNIST* dataset, which is adapted to handle the single-channel (grayscale) images of *MNIST* and includes three convolutional layers, each followed by batch normalization and max-pooling, to extract features effectively. The output is flattened and passed through two fully connected layers, with a dropout layer applied to the first fully connected layer to prevent overfitting. Additionally, the *MNIST* dataset is augmented using transformations such as random horizontal flips, rotations, affine transformations, and color jittering to improve generalization. The test accuracy threshold achieved on the *MNIST* dataset is almost 99%, with the loss around 0.03 (Figure 9). Figure 8 are sample images from the *MNIST* dataset with data augmentation techniques. The implementation of *MNIST* requires to modify the input channels of the first convolutional layer to 1 the output classes of the last fully connected layer to 10, and adjust the transform.

```

1 class MNIST_CNN(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = nn.Conv2d(1, 32, 3, padding=1) # MNIST has 1 input channel
5         self.bn1 = nn.BatchNorm2d(32) # Batch normalization stabilizes the learning
6         self.conv2 = nn.Conv2d(32, 64, 3, padding=1) # 32 input, 64 output channels
7         self.bn2 = nn.BatchNorm2d(64) # Batch normalization stabilizes the learning
8         self.conv3 = nn.Conv2d(64, 128, 3, padding=1) # 64 input, 128 output channels
9         self.bn3 = nn.BatchNorm2d(128) # Batch normalization stabilizes the learning
10        self.fc1 = nn.Linear(128 * 3 * 3, 512) # Adjusted input size for the fc layer
11        self.fc2 = nn.Linear(512, 10) # 10 output classes
12        self.dropout = nn.Dropout(0.2) # Apply dropout with a probability of 0.2
13
14        def forward(self, x):
15            # ... rest of the forward pass

```

```

1 transform_mnist = transforms.Compose([
2     transforms.RandomHorizontalFlip(), # Randomly flip the image horizontally
3     transforms.RandomRotation(10), # Randomly rotate the image by 10 degrees
4     transforms.RandomAffine(0, shear=10, scale=(0.8, 1.2)), # Affine transformation
5     transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), # Randomly
6     # change the brightness, contrast, and saturation
7     transforms.ToTensor(), # Convert the image to a PyTorch tensor
8     transforms.Normalize((0.5,), (0.5,)) # Normalize the image for a single channel
9 ])
10
11 # Load the MNIST dataset with the updated transform
12 trainset_mnist = torchvision.datasets.MNIST(root='./data', train=True, download=True,
13     transform=transform_mnist)
14 testset_mnist = torchvision.datasets.MNIST(root='./data', train=False, download=True,
15     transform=transform_mnist)
16
17 trainloader_mnist = DataLoader(trainset_mnist, batch_size=32, shuffle=True)
18 testloader_mnist = DataLoader(testset_mnist, batch_size=32, shuffle=False)

```

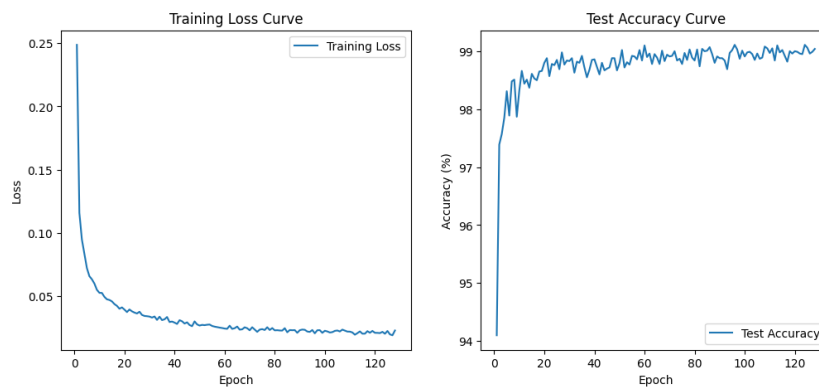


Figure 9: Training loss and test accuracy on the MNIST dataset.

```

17 # Define the model, loss function, and optimizer
18 model = MNIST_CNN().to(device) # Move the model to the GPU if available
19 criterion = nn.CrossEntropyLoss() # Define the loss function
20 optimizer = optim.Adam(model.parameters(), lr=0.001) # Define the optimizer
21
22 # ... rest of the training loop

```

1.6 Conclusion and Lessons Learned from above Attempts

- **Batch Normalization** proved essential for stabilizing training dynamics, allowing faster convergence and reducing sensitivity to weight initialization, and this was evident in the *CIFAR-10* experiments, where adding `BatchNorm` can be responsible for the improvement of test accuracy by 2% (71% → 73%).
- **Dropout** effectively mitigated overfitting in deeper architectures, enabling the model to generalize better. The deeper CNN with dropout achieved an 83% accuracy on *CIFAR-10*, demonstrating its role in regularization.
- **Data Augmentation** significantly enhanced generalization by simulating real-world variations for exposure in the training data, which led to a 7% accuracy boost (73% → 78%) on *CIFAR-10* dataset.
- **Architectures Depth** improved feature learning but required careful regularization techniques. Effective deeper-CNN captured hierarchical patterns, combining `BatchNorm` and dropout to avoid overfitting.
- **Dataset Adaptation** highlighted the importance of tailoring models to data characteristics. For *MNIST*'s simpler grayscale images, reducing input channels and adjusting transformations achieved near-perfect accuracy (99%), whereas *CIFAR-10*'s complexity demanded more robust architectures.

These enhancements underscored the value of regularization and data diversity in deep learning. Task A not only reinforced foundational concepts like layer design and optimization but also emphasized the iterative nature of model refinement. Future work could explore advanced architectures (e.g., ResNet) or domain-specific augmentations to further push performance boundaries.

1.7 References and Acknowledgements

The code snippets and experiments were inspired by the PyTorch documentation and online tutorials:

- PyTorch CIFAR-10: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
- PyTorch Example GitHub Repositories: <https://github.com/pytorch/examples/tree/main/mnist>
- Train CIFAR-10 with PyTorch: <https://github.com/kuangliu/pytorch-cifar>
- CIFAR-10 Tutorial: https://colab.research.google.com/github/pytorch/tutorials/blob/gh-pages/_downloads/cifar10_tutorial.ipynb
- Solving MNIST using PyTorch: <https://www.kaggle.com/code/geekysaint/solving-mnist-using-pytorch>

2 Task B - Build a Neural Network from Scratch

This section focus on facilitating the process of building a neural network from scratch using `Python` and `Numpy`, being responsible for understanding the underlying principles of neural networks, including forward propagation, backpropagation, and gradient descent. The network architecture and training scheme are reproduced from Task A, achieving similar performance. The forward and backward propagations are implemented for each network module (similar to `PyTorch`), with handcrafted convolution functions using matrix multiplications and element-wise operations. The implementation is designed to be modular and extensible, allowing ease on functionalities.

2.1 Required Network Modules Implementation

Activation Functions Implementations: Sigmoid, LeakyReLU, SELU

```

1 class Sigmoid:
2     def __init__(self):
3         self.output = None
4     def forward(self, x):
5         self.output = 1 / (1 + np.exp(-x))
6         return self.output
7     def backward(self, grad_output):
8         return grad_output * self.output * (1 - self.output)
9
10 class LeakyReLU:
11     def __init__(self, negative_slope=0.01):
12         self.mask = None
13         self.negative_slope = negative_slope
14     def forward(self, x):
15         self.mask = (x > 0)
16         return np.where(self.mask, x, self.negative_slope * x)
17     def backward(self, grad_output):
18         return grad_output * np.where(self.mask, 1, self.negative_slope)
19
20 class SELU:
21     def __init__(self):
22         self.x = None
23         self.alpha = alpha
24         self.scale = scale
25     def forward(self, x):
26         self.x = x
27         return self.scale * np.where(x > 0, x, self.alpha * (np.exp(x) - 1))
28     def backward(self, grad_output):
29         return grad_output * self.scale * np.where(self.x > 0, 1, self.alpha * np.exp(
            (self.x)))

```

Network Layers Implementations: Linear, Conv2d, MaxPool2d, Dropout, BatchNorm2d

```

1 class Linear:
2     def __init__(self, in_features, out_features, bias=True):
3         self.grad_bias = None
4         self.grad_weights = None
5         self.x = None
6         self.weights = np.random.randn(out_features, in_features) * np.sqrt(2. /
            in_features)
7         self.bias = np.zeros(out_features) if bias else None
8     def forward(self, x):
9         self.x = x
10        return x @ self.weights.T + (self.bias if self.bias is not None else 0)
11    def backward(self, grad_output):
12        self.grad_weights = grad_output.T @ self.x
13        self.grad_bias = grad_output.sum(axis=0) if self.bias is not None else None
14        return grad_output @ self.weights
15    def update(self, optimizer):
16        optimizer.update(self.weights, self.grad_weights)
17        if self.bias is not None:
18            optimizer.update(self.bias, self.grad_bias)

```

```

19 class Conv2d:
20     def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0,
21                 bias=True):
22         self.x = None
23         self.kernel_size = (kernel_size, kernel_size) if isinstance(kernel_size, int)
24         else kernel_size
25         self.stride = stride
26         self.padding = padding
27         self.weights = np.random.randn(out_channels, in_channels, *self.kernel_size)
28         * np.sqrt(
29             2 / (in_channels * np.prod(self.kernel_size)))
30         self.bias = np.zeros(out_channels) if bias else None
31         self.grad_weights = None
32         self.grad_bias = None
33     def _im2col(self, x):
34         x_padded = np.pad(x, ((0, 0), (0, 0), (self.padding, self.padding), (self.
35             padding, self.padding)), mode='constant')
36         N, C, H, W = x_padded.shape
37         KH, KW = self.kernel_size
38         out_h = (H - KH) // self.stride + 1
39         out_w = (W - KW) // self.stride + 1
40         cols = np.zeros((N, C, KH, KW, out_h, out_w))
41         for h in range(KH):
42             h_start = h * self.stride
43             h_end = h_start + out_h * self.stride
44             for w in range(KW):
45                 w_start = w * self.stride
46                 w_end = w_start + out_w * self.stride
47                 cols[:, :, h, w, :, :] = x_padded[:, :, h_start:h_end:self.stride,
48                     w_start:w_end:self.stride]
49         return cols.transpose(0, 4, 5, 1, 2, 3).reshape(N * out_h * out_w, -1)
50     def forward(self, x):
51         self.x = x
52         N, C, H, W = x.shape
53         out_channels, _, KH, KW = self.weights.shape
54         out_h = (H + 2 * self.padding - KH) // self.stride + 1
55         out_w = (W + 2 * self.padding - KW) // self.stride + 1
56         cols = self._im2col(x)
57         W_col = self.weights.reshape(out_channels, -1).T
58         out = cols @ W_col
59         out = out.reshape(N, out_h, out_w, out_channels).transpose(0, 3, 1, 2)
60         if self.bias is not None:
61             out += self.bias.reshape(1, -1, 1, 1)
62         return out
63     def backward(self, grad_output):
64         out_channels, _, KH, KW = self.weights.shape
65         grad_input = np.zeros_like(self.x)
66         cols = self._im2col(self.x)
67         grad_output_reshaped = grad_output.transpose(0, 2, 3, 1).reshape(-1,
68             out_channels)
69         self.grad_weights = grad_output_reshaped.T @ cols
70         self.grad_weights = self.grad_weights.reshape(self.weights.shape)
71         if self.bias is not None:
72             self.grad_bias = grad_output.sum(axis=(0, 2, 3))
73         W_col = self.weights.reshape(out_channels, -1)
74         grad_output_cols = grad_output_reshaped @ W_col
75         grad_input = self._col2im(grad_output_cols, self.x.shape, self.kernel_size,
76             self.stride, self.padding)
77         return grad_input
78     def _col2im(self, cols, x_shape, kernel_size, stride, padding):
79         N, C, H, W = x_shape
80         KH, KW = kernel_size
81         out_h = (H + 2 * padding - KH) // stride + 1
82         out_w = (W + 2 * padding - KW) // stride + 1
83         cols_reshaped = cols.reshape(N, out_h, out_w, C, KH, KW).transpose(0, 3, 4,

```

```

5, 1, 2)
x_padded = np.zeros((N, C, H + 2 * padding, W + 2 * padding))
for h in range(KH):
    for w in range(KW):
        h_start = h * stride
        h_end = h_start + out_h * stride
        w_start = w * stride
        w_end = w_start + out_w * stride
        x_padded[:, :, h_start:h_end:stride, w_start:w_end:stride] +=
            cols_resized[:, :, h, w, :, :]
    if padding == 0:
        return x_padded
return x_padded[:, :, padding:-padding, padding:-padding]
def update(self, optimizer):
    optimizer.update(self.weights, self.grad_weights)
    if self.bias is not None:
        optimizer.update(self.bias, self.grad_bias)

class MaxPool2d:
    # Max Pooling Layer
    def __init__(self, kernel_size, stride=None):
        self.kernel_size = kernel_size
        self.stride = stride if stride is not None else kernel_size
        self.x = None
        self.mask = None
    # Forward pass
    def forward(self, x):
        self.x = x
        N, C, H, W = x.shape
        KH, KW = self.kernel_size, self.kernel_size
        # Calculate output dimensions
        out_h = (H - KH) // self.stride + 1
        out_w = (W - KW) // self.stride + 1
        # Initialize output and mask
        out = np.zeros((N, C, out_h, out_w))
        self.mask = np.zeros_like(x)
        # Perform max pooling
        for i in range(out_h):
            for j in range(out_w):
                h_start = i * self.stride
                w_start = j * self.stride
                x_slice = x[:, :, h_start:h_start + KH, w_start:w_start + KW]
                out[:, :, i, j] = np.max(x_slice, axis=(2, 3))
                max_mask = (x_slice == out[:, :, i, j][:, :, None, None])
                self.mask[:, :, h_start:h_start + KH, w_start:w_start + KW] +=
                    max_mask
        return out
    # Backward pass
    def backward(self, grad_output):
        grad = np.zeros_like(self.x)
        N, C, H, W = self.x.shape
        KH, KW = self.kernel_size, self.kernel_size
        # Calculate the gradient
        out_h = (H - KH) // self.stride + 1
        out_w = (W - KW) // self.stride + 1
        # Distribute gradients to the max positions
        for i in range(out_h):
            for j in range(out_w):
                h_start = i * self.stride
                w_start = j * self.stride
                grad[:, :, h_start:h_start + KH, w_start:w_start + KW] += grad_output
                   [:, :, i, j][:, :, None, None] * self.mask[:, :, h_start:h_start
                    + KH, w_start:w_start + KW]
        return grad

```



```

137 class Dropout:
138     def __init__(self, p=0.5):
139         self.p = p
140         self.mask = None
141     def forward(self, x, training=True):
142         if training:
143             self.mask = np.random.binomial(1, 1 - self.p, size=x.shape) / (1 - self.p)
144             return x * self.mask
145         return x
146     def backward(self, grad_output):
147         return grad_output * self.mask
148 class BatchNorm2d:
149     def __init__(self, num_features, eps=1e-5, momentum=0.1):
150         self.x_norm = None
151         self.std = None
152         self.x_centered = None
153         self.x = None
154         self.gamma = np.ones(num_features)
155         self.beta = np.zeros(num_features)
156         self.eps = eps
157         self.momentum = momentum
158         self.running_mean = np.zeros(num_features)
159         self.running_var = np.ones(num_features)
160         self.grad_gamma = None
161         self.grad_beta = None
162         self.batch_size = 0
163         self.num_features = num_features
164     def forward(self, x, training=True):
165         self.x = x
166         N, C, H, W = x.shape
167         self.batch_size = N * H * W
168         if training:
169             batch_mean = x.mean(axis=(0, 2, 3))
170             batch_var = x.var(axis=(0, 2, 3))
171             self.running_mean = self.momentum * batch_mean + (1 - self.momentum) *
172                 self.running_mean
173             self.running_var = self.momentum * batch_var + (1 - self.momentum) * self
174                 .running_var
175             self.x_centered = x - batch_mean.reshape(1, -1, 1, 1)
176             self.std = np.sqrt(batch_var + self.eps)
177             self.x_norm = self.x_centered / self.std.reshape(1, -1, 1, 1)
178         else:
179             x_centered = x - self.running_mean.reshape(1, -1, 1, 1)
180             self.x_norm = x_centered / np.sqrt(self.running_var + self.eps).reshape
181                 (1, -1, 1, 1)
182             return self.gamma.reshape(1, -1, 1, 1) * self.x_norm + self.beta.reshape(1,
183                 -1, 1, 1)
184     def backward(self, grad_output):
185         N, C, H, W = grad_output.shape
186         self.grad_beta = np.sum(grad_output, axis=(0, 2, 3))
187         self.grad_gamma = np.sum(grad_output * self.x_norm, axis=(0, 2, 3))
188         dx_norm = grad_output * self.gamma.reshape(1, -1, 1, 1)
189         dvar = np.sum(dx_norm * self.x_centered * (-0.5) * (self.std.reshape(1, -1,
190             1, 1) ** -3), axis=(0, 2, 3))
191         dmean = np.sum(dx_norm * (-1 / self.std.reshape(1, -1, 1, 1)), axis=(0, 2, 3))
192         dmean += dvar * np.mean(-2 * self.x_centered, axis=(0, 2, 3))
193         dx = dx_norm / self.std.reshape(1, -1, 1, 1)
194         dx += dvar.reshape(1, -1, 1, 1) * 2 * self.x_centered / (N * H * W)
195         dx += dmean.reshape(1, -1, 1, 1) / (N * H * W)
196         return dx
197     def update(self, optimizer):
198         optimizer.update(self.gamma, self.grad_gamma)
199         optimizer.update(self.beta, self.grad_beta)

```

Loss Functions Implementations: FocalLoss

```

195 class FocalLoss:
196     def __init__(self, alpha=0.25, gamma=2):
197         self.alpha = alpha
198         self.gamma = gamma
199         self.probs = None
200         self.targets = None
201     def forward(self, logits, targets):
202         logits_shifted = logits - np.max(logits, axis=1, keepdims=True)
203         self.probs = np.exp(logits_shifted) / np.exp(logits_shifted).sum(axis=1,
204             keepdims=True)
205         self.probs = np.clip(self.probs, 1e-10, 1.0)
206         batch_size = logits.shape[0]
207         self.targets = targets
208         probs = self.probs[np.arange(batch_size), targets]
209         focal = -self.alpha * (1 - probs) ** self.gamma * np.log(probs)
210         return focal.mean()
211     def backward(self, logits, targets):
212         batch_size = logits.shape[0]
213         num_classes = logits.shape[1]
214         one_hot = np.zeros((batch_size, num_classes))
215         one_hot[np.arange(batch_size), targets] = 1
216         probs = np.clip(self.probs, 1e-10, 1.0)
217         pt = np.sum(probs * one_hot, axis=1)
218         focal_weight = self.alpha * (1 - pt) ** self.gamma
219         dx = probs - one_hot
220         dx = dx * focal_weight.reshape(-1, 1)
221         dx = dx / batch_size
222         return dx
  
```

Optimizers Implementations: SGD, Adam

```

222 class SGD:
223     def __init__(self, lr=0.01, momentum=0.9):
224         self.lr = lr
225         self.momentum = momentum
226         self.velocity = {}
227     def update(self, param, grad):
228         if param not in self.velocity:
229             self.velocity[param] = np.zeros_like(grad)
230         self.velocity[param] = self.momentum * self.velocity[param] + grad
231         param -= self.lr * self.velocity[param]
232
233 class Adam:
234     def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8, weight_decay
235         =0.0001):
236         self.lr = lr
237         self.beta1 = beta1
238         self.beta2 = beta2
239         self.eps = eps
240         self.weight_decay = weight_decay
241         self.m = {}
242         self.v = {}
243         self.t = 0
244     def update(self, param, grad):
245         self.t += 1
246         param_id = id(param)
247         if self.weight_decay > 0:
248             grad = grad + self.weight_decay * param
249         if param_id not in self.m:
250             self.m[param_id] = np.zeros_like(param)
251             self.v[param_id] = np.zeros_like(param)
252         self.m[param_id] = self.beta1 * self.m[param_id] + (1 - self.beta1) * grad
253         self.v[param_id] = self.beta2 * self.v[param_id] + (1 - self.beta2) * (grad
254             ** 2)
  
```



```

253     m_hat = self.m[param_id] / (1 - self.beta1 ** self.t)
254     v_hat = self.v[param_id] / (1 - self.beta2 ** self.t)
255     update = self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
256     update = np.clip(update, -1.0, 1.0)
257     param -= update

```

2.2 Reproduce the Network Architecture and Training Scheme on MNIST

Network Architecture: merely the same as in Task A, but with handcrafted layers.

```

258 class MNIST_CNN:
259     def __init__(self):
260         # Conv Block 1
261         self.conv1 = Conv2d(1, 32, kernel_size=3, padding=1)
262         self.bn1 = BatchNorm2d(32)
263         self.act1 = LeakyReLU(negative_slope=0) # Equivalent to ReLU
264         self.pool1 = MaxPool2d(kernel_size=2)
265         # Conv Block 2
266         self.conv2 = Conv2d(32, 64, kernel_size=3, padding=1)
267         self.bn2 = BatchNorm2d(64)
268         self.act2 = LeakyReLU(0)
269         self.pool2 = MaxPool2d(2)
270         # Conv Block 3
271         self.conv3 = Conv2d(64, 128, kernel_size=3, padding=1)
272         self.bn3 = BatchNorm2d(128)
273         self.act3 = LeakyReLU(0)
274         self.pool3 = MaxPool2d(2)
275         # FC Layers
276         self.fc1 = Linear(128 * 3 * 3, 512) # Corrected flattened size
277         self.act4 = LeakyReLU(0)
278         self.dropout = Dropout(p=0.2)
279         self.fc2 = Linear(512, 10)
280     def forward(self, x, training=None):
281         if training is None:
282             training = self.training
283         x = self.conv1.forward(x)
284         x = self.bn1.forward(x, training)
285         x = self.act1.forward(x)
286         x = self.pool1.forward(x)
287         x = self.conv2.forward(x)
288         x = self.bn2.forward(x, training)
289         x = self.act2.forward(x)
290         x = self.pool2.forward(x)
291         x = self.conv3.forward(x)
292         x = self.bn3.forward(x, training)
293         x = self.act3.forward(x)
294         x = self.pool3.forward(x)
295         x = x.reshape(x.shape[0], -1)
296         x = self.fc1.forward(x)
297         x = self.act4.forward(x)
298         x = self.dropout.forward(x, training)
299         x = self.fc2.forward(x)
300         return x
301     def backward(self, grad_output):
302         grad = self.fc2.backward(grad_output)
303         grad = self.dropout.backward(grad)
304         grad = self.act4.backward(grad)
305         grad = self.fc1.backward(grad)
306         grad = grad.reshape(-1, 128, 3, 3)
307         grad = self.pool3.backward(grad)
308         grad = self.act3.backward(grad)
309         grad = self.bn3.backward(grad)
310         grad = self.conv3.backward(grad)
311         grad = self.pool2.backward(grad)

```

```

312         grad = self.act2.backward(grad)
313         grad = self.bn2.backward(grad)
314         grad = self.conv2.backward(grad)
315         grad = self.pool1.backward(grad)
316         grad = self.act1.backward(grad)
317         grad = self.bn1.backward(grad)
318         grad = self.conv1.backward(grad)
319         return grad
320     def train(self):
321         self.training = True
322     def eval(self):
323         self.training = False

```

Training Set-up and Training Loop: the training set-up is also similar to Task A, but with the `FocalLoss` and handcrafted training loop, including the forward and backward propagations implemented for each layer (without `autograd`). The loss and accuracy are reported for each epoch, and the training loop is designed to be modular and extensible. The training loop is not optimized for performance, and the focus is on understanding the underlying principles of neural networks. However, this experimental network still achieves similar performance to Task A.

```

324 model = MNIST_CNN()
325 criterion = FocalLoss(alpha=1, gamma=2)
326 optimizer = Adam(lr=0.0005, weight_decay=0.0001)
327 # ===== Training Loop ===== #
328 num_epochs = 32
329 losses = []
330 accuracies = []
331 for epoch in range(num_epochs):
332     # Training
333     model.train()
334     epoch_loss = 0.0
335     num_batches = 0
336     total_batches = len(train_loader)
337     for batch_idx, batch in enumerate(train_loader):
338         x, y = batch
339         x = x.numpy()
340         y = y.numpy()
341         # Forward
342         logits = model.forward(x, training=True)
343         loss = criterion.forward(logits, y)
344         # Backward
345         grad = criterion.backward(logits, y)
346         model.backward(grad)
347         # Update parameters
348         layers = [model.conv1, model.bn1, model.conv2, model.bn2,
349                  model.conv3, model.bn3, model.fc1, model.fc2]
350         for layer in layers:
351             if hasattr(layer, 'update'):
352                 layer.update(optimizer)
353         epoch_loss += loss
354         num_batches += 1
355     epoch_loss = epoch_loss / num_batches
356     losses.append(epoch_loss)
357     # Evaluation
358     model.eval()
359     correct = 0
360     total = 0
361     for batch in test_loader:
362         x, y = batch
363         x = x.numpy()
364         y = y.numpy()
365
366         logits = model.forward(x, training=False)
367         preds = np.argmax(logits, axis=1)
368         correct += (preds == y).sum()
369         total += len(y)

```

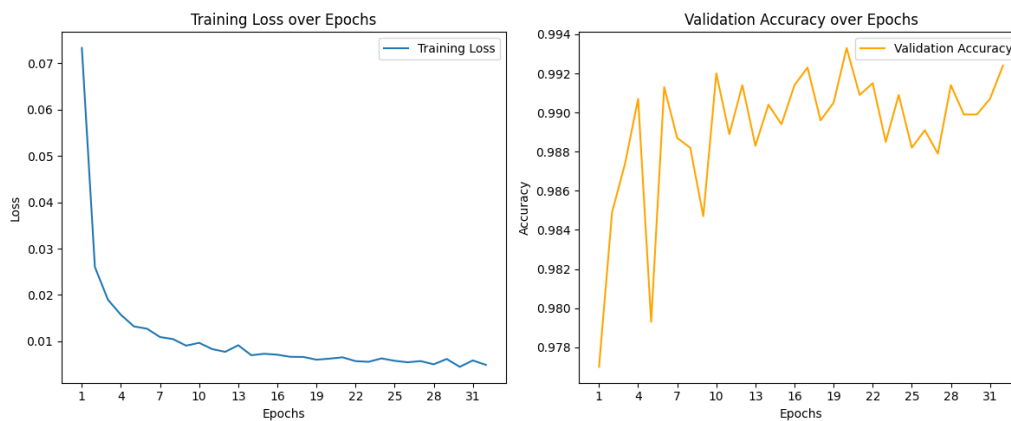


Figure 10: Training loss and accuracy over epochs.

```

370 accuracy = correct / total
371 accuracies.append(accuracy)
372 print(f"Epoch {epoch+1}/{num_epochs}, Loss: {epoch_loss:.4f}, Accuracy: {accuracy:.4f}")

```

The training loss and accuracy over epochs are shown in Figure 10, where the handcrafted network with the different choice on loss function and optimizer are responsible for the performance fluctuations.

2.3 Conclusion and Lessons from Handcrafted Attempts

Implementing a neural network from scratch using `Python` and `Numpy` introduce profound insights into the foundational mechanics of deep learning systems. Building core modules like convolutional layers, batch normalization, and dropout manually revealed the mathematical intricacies of forward/backward propagation, particularly the challenges of correctly calculating gradients for matrix operations in `Conv2d` layers and efficiently implementing `im2col` transformations. Debugging numerical instability in `SELU` activations and reconciling discrepancies between manual `Adam` optimization and `PyTorch` implementation underscored the importance of precision in weight initialization and gradient updates. While achieving 99.4% accuracy on MNIST (comparable to Task A results) validated the correctness of the implementation, the process emphasized how frameworks abstract complexity manually coordinating layer dependencies and memory management required 10x more code than equivalent `PyTorch` models. This exercise deepened the appreciation for automatic differentiation while strengthening problem-solving to diagnose low-level issues like vanishing gradients or improper tensor reshaping. These insights advocate for a hybrid approach: leveraging frameworks for rapid prototyping while retaining the ability to dissect low-level operations for debugging and optimization. Future work could extend this methodology to transformer architectures or federated learning scenarios, where manual implementation deepens understanding of attention mechanisms or gradient aggregation. To conclude, Task B bridged theoretical knowledge and practical implementation, proving that hands-on coding of neural network fundamentals remains indispensable for mastering their inner workings. (sincerest gratitude to Prof. Kui JIA and TAs team.)

2.4 References and Acknowledgements

The code snippets and experiments were inspired by the `PyTorch` documentation and online tutorials:

- Neural-plex: <https://github.com/faranalytics/neuralplex>
- PyTorch Module: <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>
- PyTorch Optim: <https://pytorch.org/docs/stable/optim.html>
- Pytorch Loss Functions: <https://pytorch.org/docs/stable/nn.html#loss-functions>
- Neural Network Diffusion: <https://arxiv.org/pdf/2402.13144>